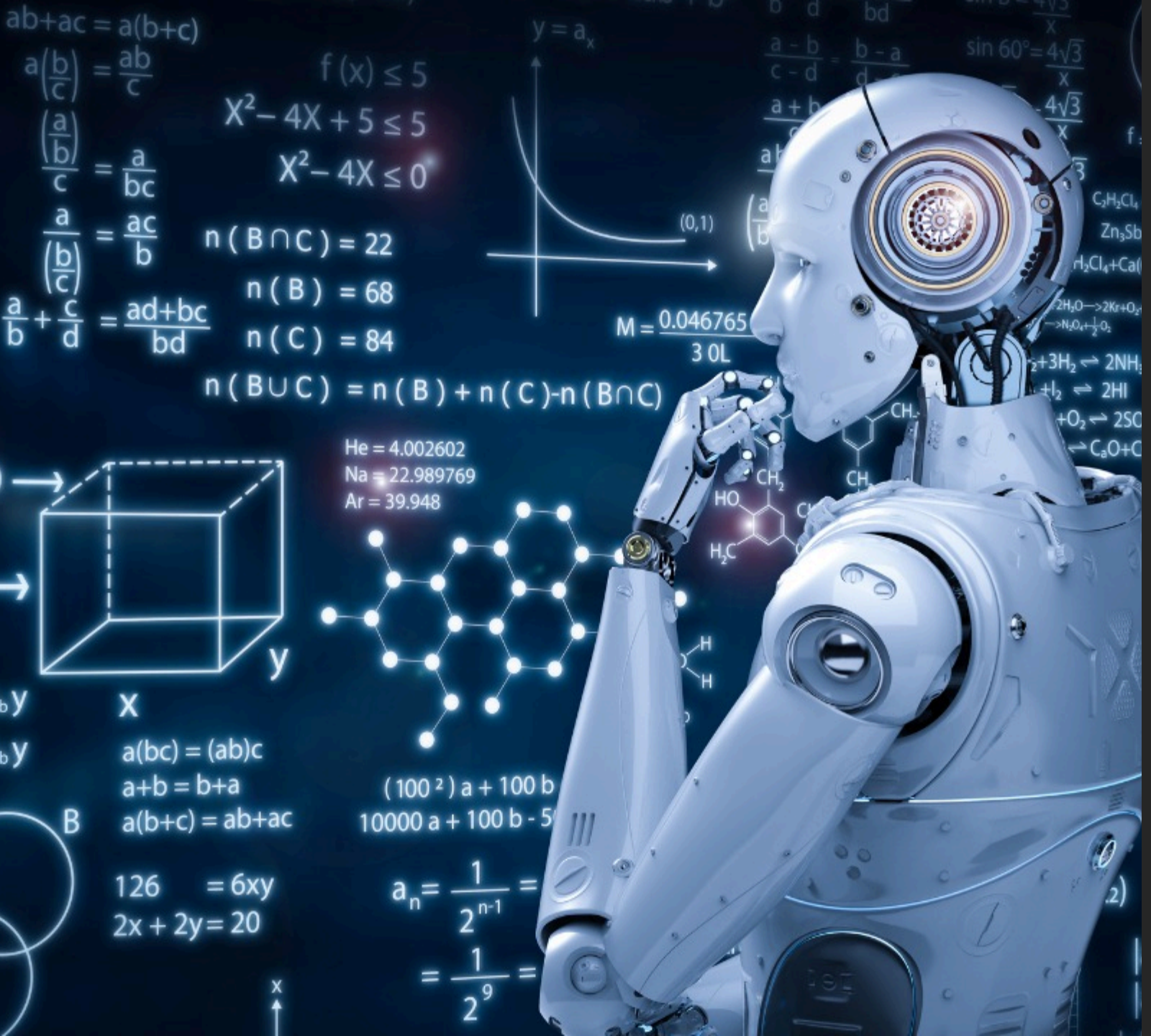


Learning Outcome 4

RECURSION AND GRAPH TRAVERSALS

PROF. DR. RASHMI GUPTA

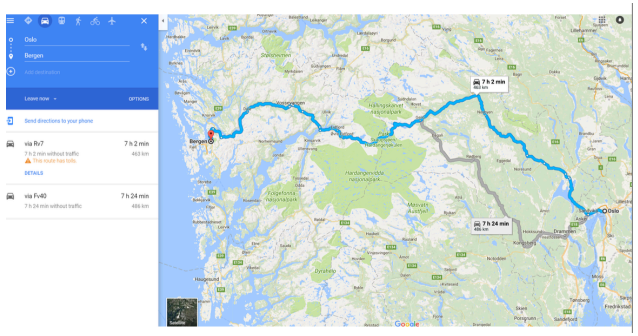
RASHMI.GUPTA@KRISTIANIA.NO

Major Objectives

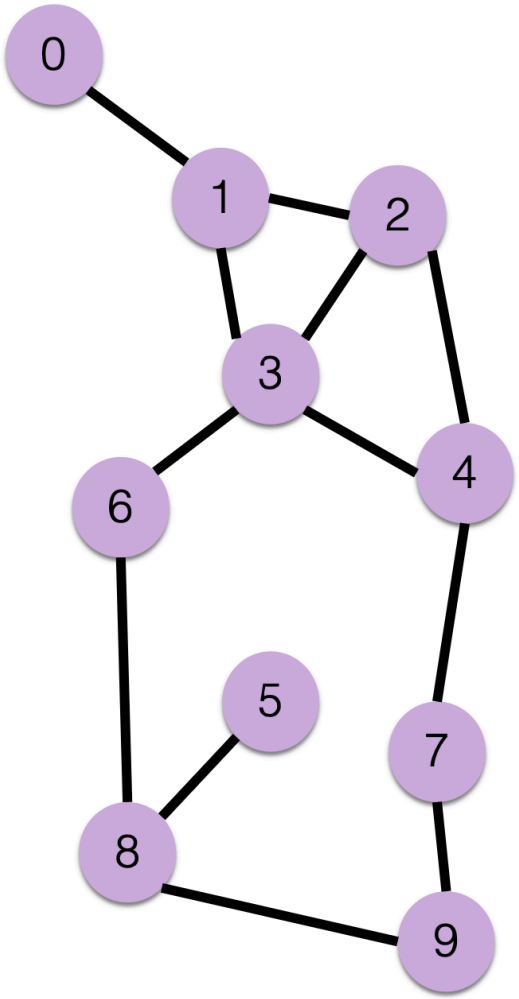
- Today´s Agenda
  - Graph Data Structures and Graph Traversal Algorithms
    - Breadth-First Search (BFS)
    - Depth-First Search (DFS)

Recap: Graphs

- ✓A set of vertices connected by edges
- ✓Directed or Undirected graphs
  - ✓ If edge from X to Y implies an edge from Y to X?
- ✓Many different problems can be represented with graphs
- ✓Many different algorithms specialized for graphs

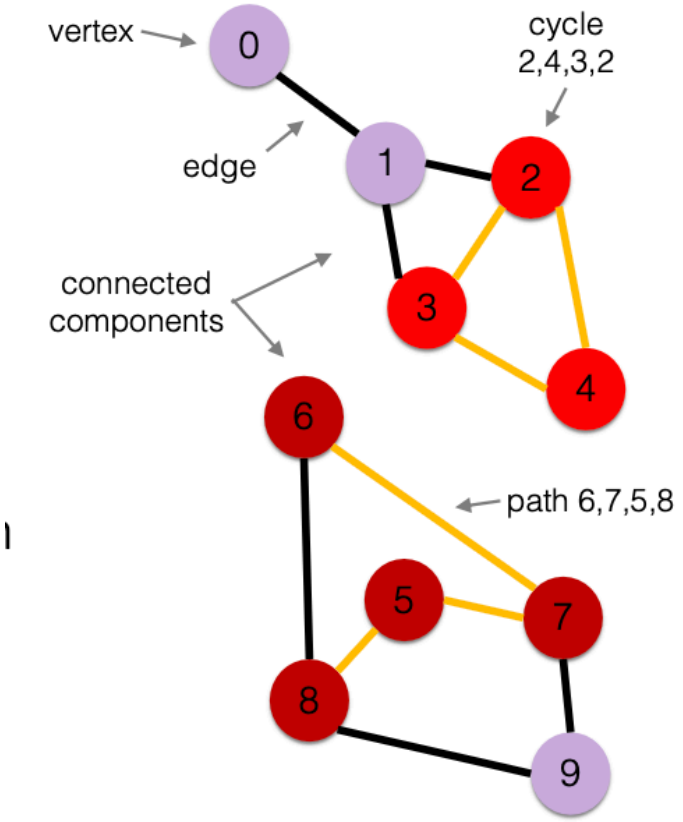


Friends in a social network



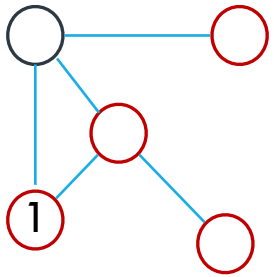
Recap: Graph Terminology

- ✓**Vertex:** a node, for which can use a label to identify it
- ✓**Edge:** connection between 2 nodes
- ✓**Path:** a sequence of connected nodes
- ✓**Cycle:** a path starting and ending on the same node
- ✓**Note:** in a graph, not all nodes are necessarily connected



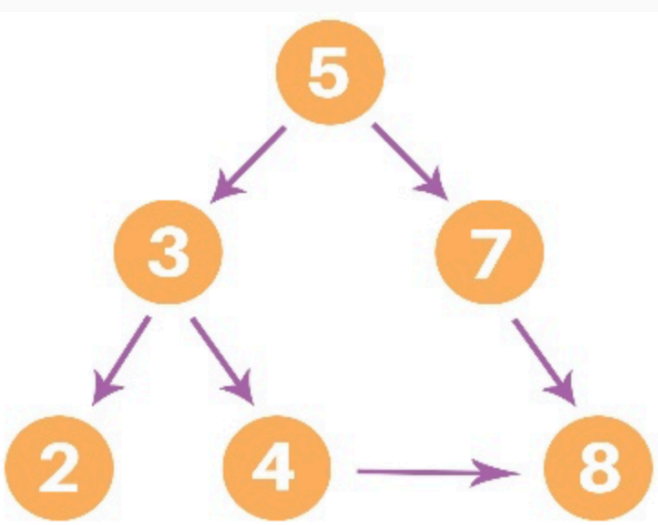
Depth First Search (DFS)

- A standard DFS implementation puts each vertex of the graph into one of two categories:
  - Visited
  - Not Visited
- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.
- The DFS algorithm works as follows:
  - Start by putting any one of the graph's vertices on top of a stack.
  - Take the top item of the stack and add it to the visited list.
  - Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
  - Keep repeating steps 2 and 3 until the stack is empty. Java and python implementation. HERE

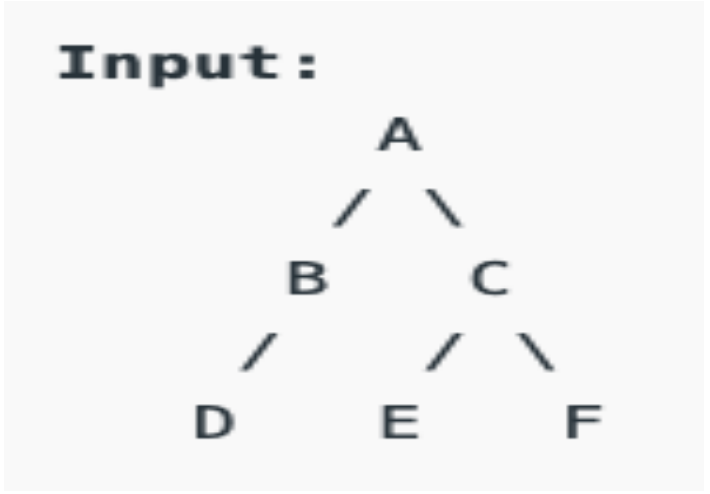


Other possible output: 1 4 2 3 0

Let´s do some practice!!!



Output: 7, 8, 4, 2, 3, 5
2, 4, 3, 8, 7, 5



Output (DFS): D B E F C A

E F C D B A

Shall we try **DFS Search on these Graphs!!**

Additional Information: DFS

✓Complexity of Depth First Search

- ✓ The **time complexity** of the DFS algorithm is represented in the form of **O(V + E),** where V is the **number of nodes** and **E** is the **number of edges.**
- ✓ The **space complexity** of the algorithm is **O(V).**

✓Application of DFS Algorithm

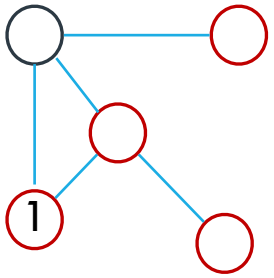
- ✓ To find the path
- ✓ For finding the strongly connected components of a graph
- ✓ For detecting cycles in a graph

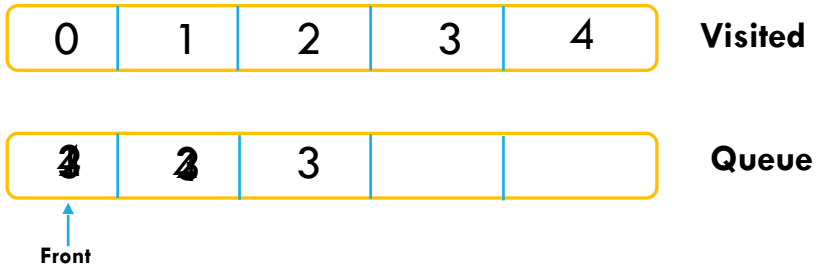
Real-Time Uses of DFS

- **Solving Mazes and Puzzles**
  - DFS is often used in **real-time puzzle games** or simulations where the goal is to find any path (not necessarily the shortest).
- **Backtracking Algorithms**
  - Examples:
    - **Sudoku solvers**
    - **Crossword or Word Search solvers**
    - **N-Queens problem** in real-time chess puzzle generators
- **Cycle Detection in Graphs**
  - Used in **real-time deadlock detection** systems in operating systems or databases.
- **Web Crawlers (Deeper Search)**
  - When the crawler is set to go deep into a site before going to the next site (depth-based exploration).

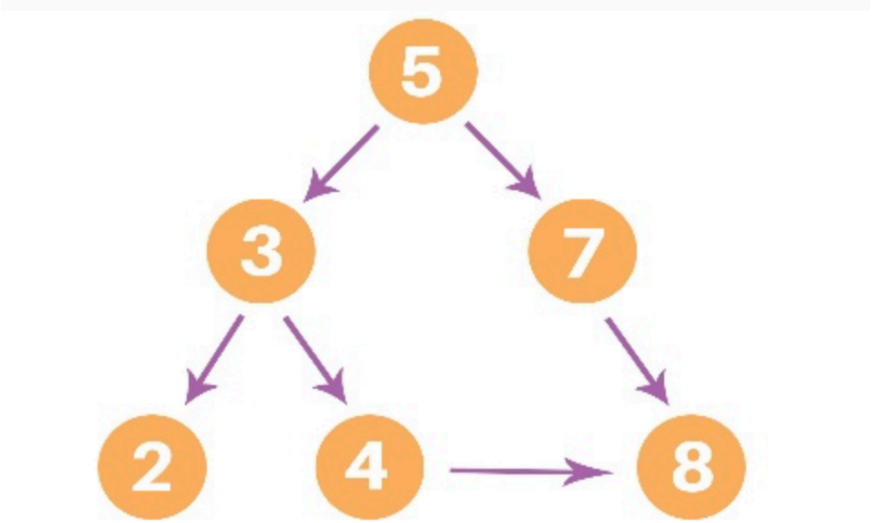
Breadth First Search (BFS)

- A standard BFS implementation puts each vertex of the graph into one of two categories:
  - Visited
  - Not Visited
- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles. The algorithm works as follows:
  - Start by putting any one of the undirected graph's vertices at the back of a queue.
  - Take the front item of the queue and add it to the visited list.
  - Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
  - Keep repeating steps 2 and 3 until the queue is empty.
  - The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node.

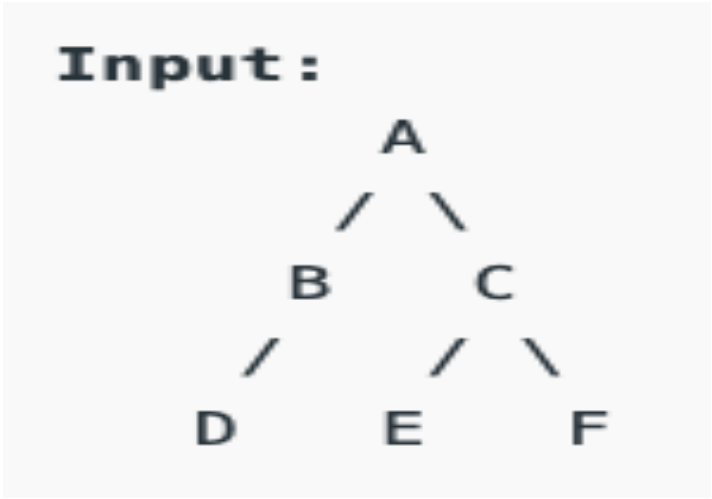




Let´s do some practice!!!



Output: 5, 3, 7, 2, 4, 8



Output: A, B, C, D, E, F

Shall we try **BFS Search on these Graphs!!**

Additional Information: BFS

✓The complexity of Breadth-First Search

- ✓ The **time complexity** of the BFS algorithm is represented in the form of **O(V + E),** where V is the **number of nodes** and **E** is the **number of edges.**
- ✓ The **space complexity** of the algorithm is **O(V).**

✓Application of DFS Algorithm

- For GPS navigation
- Pathfinding algorithms
- Cycle detection in an undirected graph
- Java and Python implementation. HERE

🏠 Example: Finding the Shortest Path in a House

```
Living Room
   /     \
Hallway  Bathroom
   |
Bedroom
   |
Kitchen
```

How DFS Works Here:
Let's assume DFS always explores the **left path first** (if multiple options are available). So from Living Room, it will go to **Hallway** before **Bathroom.**
DFS Traversal:
1.Start at **Living Room**
2.Go to **Hallway** (first neighbor)
3.Go to **Bedroom**
4.Go to **Kitchen** 🎉 — found the goal!

🧭 Path taken:
`Living Room → Hallway → Bedroom → Kitchen`

✅ DFS *also* found the kitchen — same as BFS in this case — **but not because it was looking for the shortest path.** It just happened to go the right way. DFS explores **as deep as possible** before backtracking.

How BFS Works Here:
BFS will explore **all rooms 1 step away first,** then **2 steps away,** and so on — until it finds the kitchen.
1.Start at **Living Room**
2.Visit all directly connected rooms:
   1. **Hallway**
   2. **Bathroom**
3.Now visit rooms connected to those:
   1. From Hallway → **Bedroom**
   2. From Bathroom → (nothing)
4.Next level:
   1. From Bedroom → **Kitchen**

✔️ You found the **Kitchen** in **3 steps:**
`Living Room → Hallway → Bedroom → Kitchen`
BFS is great here because it finds the **shortest path (fewest rooms)** to your goal — the kitchen — without going deep in the wrong direction (like Bathroom → dead end)